



Queues – FIFO in Everyday Life and Python

Simple, fair, and everywhere. This short walk-through shows what a queue is, real-world examples, why Python lists are inefficient for queues, and how `collections.deque` solves the problem.



Real life hook: Why the printer prints the first document first

The printer serves jobs in the order they're received: First In → First Out (FIFO). This ordering is the defining behavior of a queue — the earliest request is the first to be handled.

What is a Queue?

A queue is a linear data structure with two primary operations:

- Enqueue: insert at the BACK (rear)
- Dequeue: remove from the FRONT

Rule: FIFO — First In, First Out.



Real-World Examples



People standing in line

Customers are served in arrival order — fairness and predictability.



Printer job scheduling

Documents print in order received to avoid confusion and collisions.



Food delivery system

Orders are prepared and handed out in sequence to ensure correct fulfillment.



Customer care call system

Callers wait in queue so the earliest caller is connected first.



CPU task scheduling

Some scheduling strategies treat tasks like queues for fairness and throughput.

Quick takeaway: Queue semantics are commonly used wherever fairness and ordered processing matter.

Example:

Problem: `pop()` removes the first element but shifts all remaining elements left. That shifting is $O(n)$ — slow for large queues. We need a structure that supports fast front and back operations.

Introducing deque

Python provides `collections.deque` — a double-ended queue optimized for fast appends and pops from both ends.

- Import it with: `from collections import deque`
- Designed for $O(1)$ operations at either end (amortized)
- Ideal for implementing FIFO queues efficiently



Creating a queue

Code:

```
from collections import deque  
queue = deque()
```

Result: an empty deque object — behaves like `[]` but optimized for both ends.



Adding elements (enqueue)

Use `append()` to add to the back:

```
queue.append("Ali")  
queue.append("Sara")  
queue.append("Ahmed")
```

Queue order (front → back): **Ali → Sara → Ahmed**. The first enqueued item (Ali) will be dequeued first.

Printing a deque

When you print a deque you see its special representation:

```
print(queue)
# deque(['Ali', 'Sara', 'Ahmed'])
```

It's not a list — but it behaves similarly for iteration and indexing. For queue semantics, prefer `popleft()` to remove from the front efficiently.

Best Practices & Quick Reference

Use deque for queues

$O(1)$ appends and pops at both ends — efficient for FIFO use.

Enqueue

`queue.append(x)` — add to the back.

Deque

`queue.popleft()` — remove from the front quickly.

Avoid `pop(0)` on lists

It's $O(n)$ and becomes a bottleneck for large queues.

Try it in a small script to see timings: for large workloads, deque is the practical choice.

